Зачем столько языков?

Плотников Даниил Михайлович

Санкт-Петербургский Государственный Университет

Сколько языков существует?

- Согласно TIOBE index: около 250.
- Согласно HOPL: 8945 на момент 2020 года.

Feb 2026	Feb 2025	Change	Programming Language	Ratings	Change
1	1		Python	21.81%	-2.08%
2	4	^	C	11.05%	+1.22%
3	2	v	C++	8.55%	-2.82%
4	3	v	Java	8.12%	-2.54%
5	5		C#	6.83%	+2.71%
6	6		JavaScript	2.92%	-0.85%
7	10	^	Visual Basic	2.85%	+0.81%
8	15	^^	R	2.19%	+1.14%
9	**7**	v	**SQL**	**1.93%**	**-0.93%**
10	9	v	Delphi/Object Pascal	1.88%	-0.29%

Рис. 1. 10 самых популярных языков согласно индексу TIOBE

История языков программирования

Ада Лавлейс

Авгу́ста А́да Кинг(10 декабря 1815 — 27 ноября 1852) — английская графиня и математик. Прежде всего известна описанием вычислительной машины, а также первой в истории программой для неё.

Рис. 2. Авгу́ста А́да Кинг

ЭВМ — электронно-вычислительная машина

В 1940-е годы впервые появились ламповые электронно-вычислительные машины:

- Z3 (1941): Первая функциональная программируемая машина (Германия).
- Colossus (1943): Специализированная английская машина.
- ENIAC (1946): Первый универсальный электронный компьютер (США).

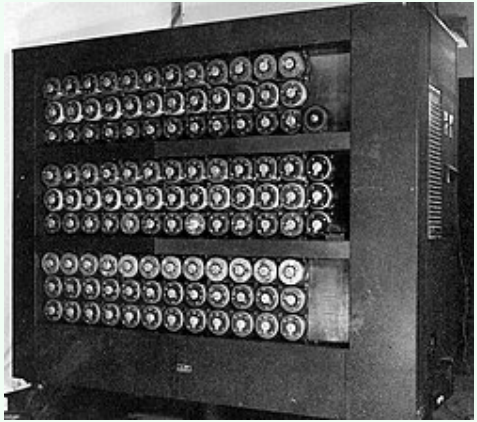

Рис. 3. Bombe

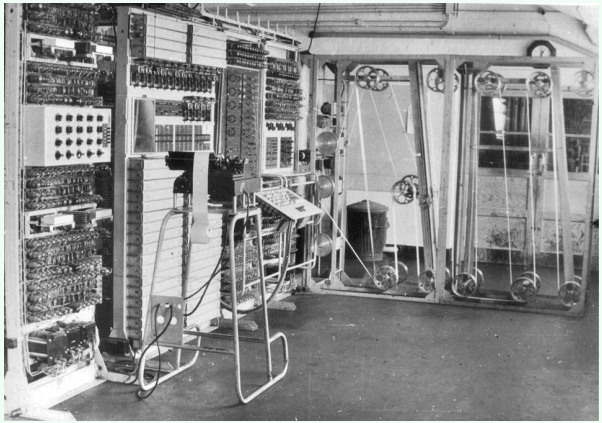

Рис. 4. Colossus Mark 2

Первый высокоуровневый язык

Plankalkül — первый в мире высокоуровневый язык программирования, созданный немецким инженером Конрадом Цузе в 1943—1945. Впервые опубликован в 1972 г., первый компилятор был выпущен в 1975 г.

A[5] = A[4]+1

	A + 1	⇒	A
V	4		5
S	$1.n$		$1.n$

,

```
P1 max3 (V0[:8.0],V1[:8.0],V2[:8.0]) → R0[:8.0]
max(V0[:8.0],V1[:8.0]) → Z1[:8.0]
max(Z1[:8.0],V2[:8.0]) → R0[:8.0]
END
P2 max (V0[:8.0],V1[:8.0]) → R0[:8.0]
V0[:8.0] → Z1[:8.0]
(Z1[:8.0] < V1[:8.0]) → V1[:8.0] → Z1[:8.0]
Z1[:8.0] → R0[:8.0]
END
```

Ассемблер

На первых компьютерах для написания программ использовали двоичный код. В конце 1940-х годов появились первые ассемблеры.

- Кэтлин Бут (1947, ARC2)
- Дэвид Уилер (1948, EDSAC)

Мнемоника

Комментарии

```
start:  mov   ax, 5     ; загрузить число 5 в регистр ax
```

Метка

Операнды

Отцы программирования

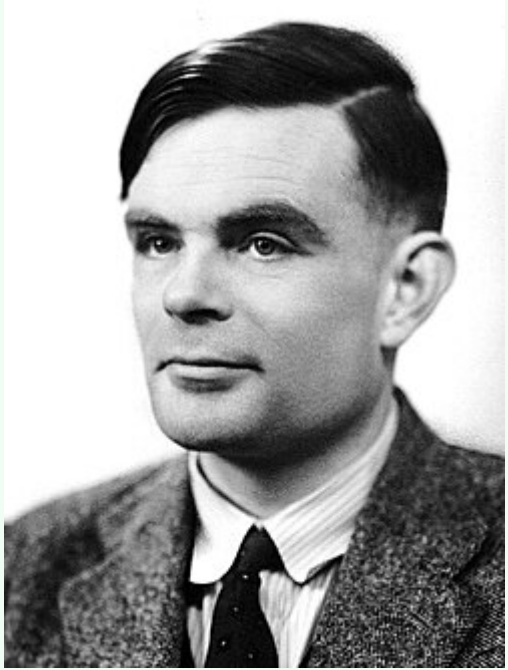

Рис. 5. Алан Тьюринг

Рис. 6. Алонзо Чёрч

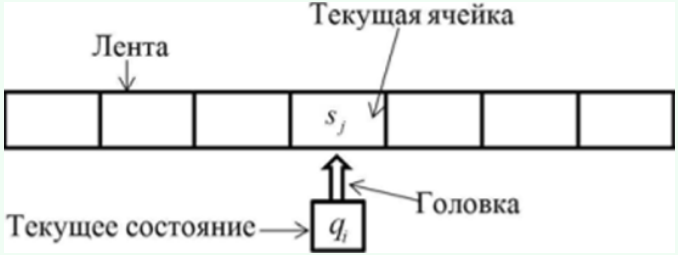

Рис. 7. Машина Тьюринга

«О вычислимых числах в приложении к проблеме разрешения» 1936 г.

$$n + m : \lambda m.\lambda n.\lambda f.\lambda x.mf(nfx)$$

$$\text{True} : \lambda x.\lambda y.x$$

$$\text{False} : \lambda x.\lambda y.y$$

$$\text{If} : \lambda b.\lambda t.\lambda e.bte$$

Рис. 8. Лямбда-исчисление

«Система постулатов для оснований логики» 1932 г.

Fortran

Fortran(**For**mula **Tran**slator) — первый реализованный высокоуровневый язык программирования, получивший широкое признание. Создан в 1957 г.

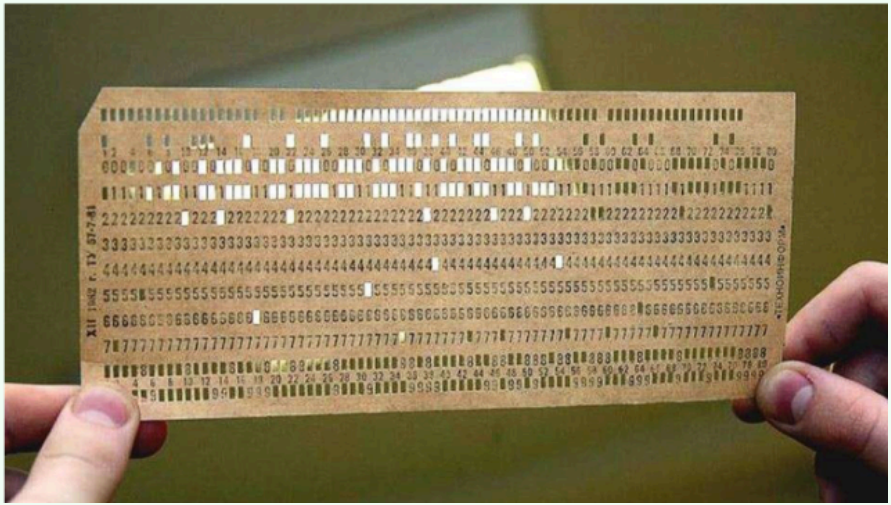

Рис. 9. Fortran использовал для программирования не текстовый язык, а перфокарты

Algol

Algol — название семейства языков программирования, применяемого для решения научно-технических задач. Первый язык семейства появился в 1958 г.

```
begin
   integer n, i, sum;
   integer array arr[1:1000];  % Use maximum size available
   sum := 0;
   read(n);
    begin
       for i := 1 step 1 until n do
          begin
             read(arr[i]);
             sum := sum + arr[i]
          end;
       print(sum)
    end
end
```

Simula

Simula — язык программирования общего назначения, созданный в 1960 г. Является объектным расширением Algol60. Именно в нём появились основные синтаксические соглашения, которые впоследствии получили название «объектно-ориентированное программирование».

```
begin
   integer n, i, sum;
   read(n);
   integer array arr[1:n];
   sum := 0;
   for i := 1 step 1 until n do
      begin
         read(arr[i]);
         sum := sum + arr[i]
      end;
   outtext("Sum = ");
   outputnum(sum)
end
```

Рис. 10. Оле-Йохан Даль и Кристен Нюгорд

Basic

BASIC(**B**eginner's **A**ll-purpose **S**ymbolic **I**nstruction **C**ode) — семейство компилируемых статически типизированных языков программирования общего назначения. Первый язык был разработан Томасом Курцем и Джоном Кемени в 1964 г.

```
10 REM DICE ROLL SIMULATOR
20 DIM ROLL(6)
30 FOR I = 1 TO 1000
40   R = INT(RND*6)+1
50   ROLL(R) = ROLL(R) + 1
60 NEXT I
70 PRINT "RESULT OF 1000 DICE ROLLS:"
80 FOR I = 1 TO 6
90   PRINT I; " : "; ROLL(I)
100 NEXT I
```

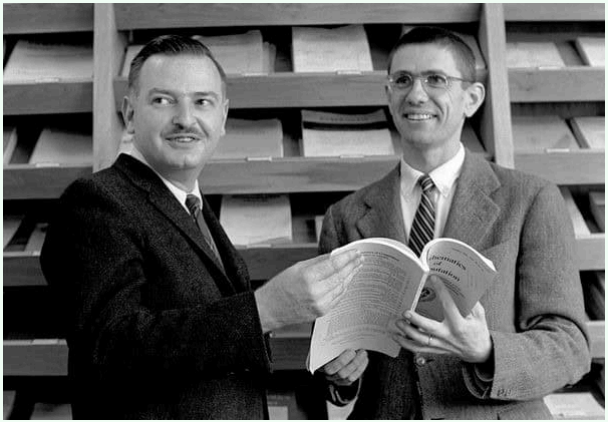

Рис. 11. Томас Курц и Джон Кемени

Pascal

Pascal — компилируемый статически типизированный язык программирования общего назначения, разработанный Никлаусом Виртом в 1970 г.

```
program SumArray(input, output);
const
  MAX_N = 10000;
var
  n, i, sum: integer;
  arr: array[1..MAX_N] of integer;
begin
  sum := 0;
  read(n);
  for i := 1 to n do
  begin
    read(arr[i]);
    sum := sum + arr[i];
  end;
  writeln('The sum of array is ', sum);
end.
```

Рис. 12. Никлаус Вирт

C

C — компилируемый статически типизированный язык программирования общего назначения, разработан Деннисом Ритчи и Кеном Томпсоном в 1972 г.

```c
#include <stdio.h>
#include <stdlib.h>

int main(){
  int n, sum = 0, *array;
  scanf("%d", &n);
  array = malloc(sizeof(int)*n);
  for(int i = 0; i < n; ++i){
    scanf("%d", array+i);
    sum += *(array+i);
  }
  printf("The sum of array is %d", sum);
  free(array);
  return 0;
}
```

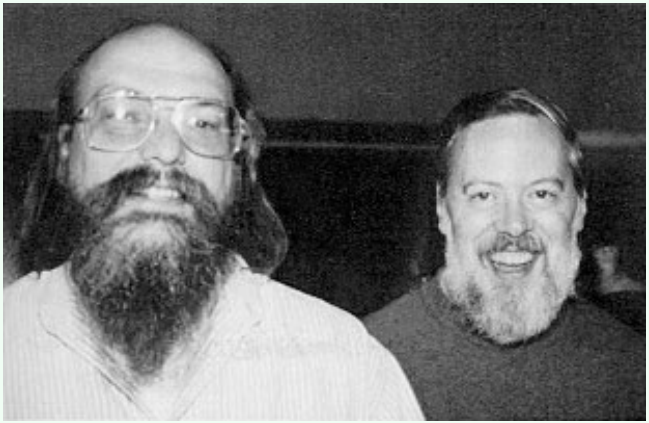

Рис. 13. Дэннис Ритчи и Кен Томпсон

C++

C++ — компилируемый статически типизированный язык программирования общего назначения, разработанный Бьёрном Страуструпом в 1979 г.

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int main(){
  int n;
  cin >> n;
  vector<int> vec(n);
  for(auto &el : vec){
    cin >> el;
  }
  cout << "The sum of array is " <<
accumulate(vec.begin(), vec.end(), 0);
  return 0;
}
```

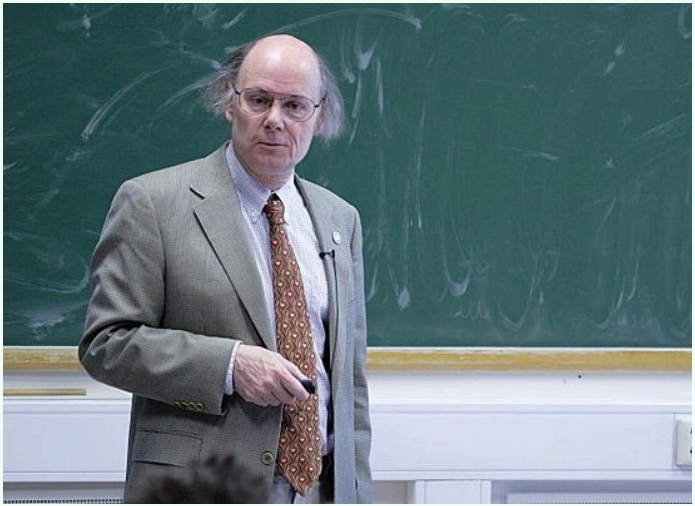

Рис. 14. Бьёрн Страуструп

Python

Python — интерпретируемый язык программирования общего назначения с динамической строгой типизацией и автоматическим управлением памятью, разработанный Гвидо ван Россумом в 1991 г.

```
import string

n = int(raw_input())
parts =
string.split(raw_input())
vec = map(int, parts)

total = 0
for num in vec:
  total = total + num

print "The sum of array is", total
```

Рис. 15. Гвидо ван Россум

Java

Java — интерпретируемый язык программирования общего назначения с статической строгой типизацией и автоматическим управлением памятью, разработанный Sun Microsystems в 1995 г.

```java
import java.util.Scanner;
public class ArraySum {
  public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    int n = scanner.nextInt();
    int[] arr = new int[n];
    for (int i = 0; i < n; i++) {
      arr[i] = scanner.nextInt();
    }
    int sum = 0;
    for (int num : arr) {
      sum += num;
    }
    System.out.println("The sum of array is " + sum);
    scanner.close();
  }
}
```

Рис. 16. Sun Microsystems

Другие языки

Новые языки появляются и по сей день, каждый из них решает свои задачи и развивается параллельно остальным. Некоторые выделяющиеся языки, о которых я ещё не упоминал:

- Системный уровень: Rust (2010), Zig (2015).
- Языки интернета: JavaScript (1995), Ruby (1995), PHP (1995).
- ООП-продолжатели Java — C# (2000), Kotlin (2011), — свой C++ для Java.
- Go (2009), Carbon (2022), Dart (2011) — языки от Google.
- Декларативные языки. Скажи **что**, а не **как** — SQL (1974), HTML (1993), Prolog (1972)

LISP

LISP — функциональный язык программирования, разработанный группой учёных из MIT в 1960 году.

```
(format t "Hello, world!~%")
; N-е число Фибоначчи
(defun fibonacci (n)
  (loop repeat n
    for a = 0 then b
    and b = 1 then (+ a b)
    finally (return b)))
```

Рис. 17. Джон Маккарти

«... Требовалась нотация для функций, и λ-нотация Чёрча (1941) казалась естественной. Остальную часть книги я не понял, так что не было желания реализовывать его более общие механизмы определения функций.» История LISP, 1978 г.

ML

ML(**M**eta **L**anguage) — функциональный язык программирования, разработанный Робином Милнером в 1970 г.

```
(* N-е число Фибоначчи *)
let rec fibonacci n =
  if n < 3 then
    1
  else
    fibonacci (n-1) + fibonacci (n-2)

let () =
  for n = 1 to 16 do
    Printf.printf "%d, " (fibonacci n)
  done;
    print_endline "..."
```

Рис. 18. Робин Милнер

Haskell

Haskell — функциональный язык программирования семейства ML, разработанный в 1990 г.

```haskell
-- Числа Фибоначчи
module Main where
import Text.Printf

fibs :: [Int]
fibs = 0 : 1 : zipWith (+) fibs (tail fibs)

line n = printf "%d, " $ fibs !! n

main = do
  sequence_ $ map line [1..16]
  putStrLn "..."
```

Другие языки

Мир функциональных языков так же активно развивается. Не только возможности функциональных языков перетекают в языки Тьюринга, но и новые чисто функциональные языки применяются на практике.

- Параллельные и отказоустойчивые: Erlang (1986), Elixir (2011)
- Гибриды (функциональный + ООП): Scala (2004), F# (2005)
- Функциональные веб-языки: Elm (2012), PureScript (2013), Rescript (2016)
- Современные системные функциональные: OCaml (1996), Clojure (2007), F# (2005)

Выбор языка

Чем языки отличаются?

Парадигма — как структурировать программу?
Императивная · Функциональная · ООП · Декларативная

Система типов — когда ловятся ошибки?
Статическая · Динамическая · Строгая · Слабая

Модель исполнения — как код становится действиями?
Компиляция · Интерпретация · JIT-компиляция

Управление памятью — кто следит за ресурсами?
Ручное · Сборщик мусора · Ownership

«КАК делать» vs «ЧТО получить»

Задача: имена отличников, отсортированные по алфавиту

Императивный (Python)

```python
result = []
for s in students:
    if s.grade >= 5:
        result.append(s.name)
result.sort()
```

Пошагово описываем процесс

Декларативный (SQL)

```sql
SELECT name
FROM students
WHERE grade >= 5
ORDER BY name;
```

Описываем желаемый результат

Статическая vs динамическая типизация

Статическая (Java)

```
int x = 5;
x = "hello";
// Ошибка при КОМПИЛЯЦИИ
// Программа не запустится
```

✓ Ошибки ловятся до запуска

✗ Нужно указывать типы

Динамическая (Python)

```
x = 5
x = "hello"
print(x + 1)
# TypeError при ЗАПУСКЕ
```

✓ Быстро прототипировать

✗ Ошибки всплывают в рантайме

Строгая vs слабая типизация

Строгая (Python)

```
>>> "5" + 3
TypeError: can only
concatenate str to str

>>> "5" * 3
"555"  # явно определено
```

Язык *отказывается* смешивать типы. Нужно явно преобразовать: `int("5") + 3`

Слабая (JavaScript)

```
> "5" + 3
"53"     // строка!

> "5" - 3
2        // число!
```

Язык *сам* преобразует типы на ходу. Результат бывает неожиданным.

От кода к результату

Компиляция (C, C++, Rust) *Быстрый результат, долгая сборка*

Исходный код → Компилятор → Маш. код → Процессор

Интерпретация (Python, Ruby) *Быстрый запуск, медленная работа*

Исходный код → Интерпретатор → Процессор

JIT-компиляция (Java, C#, JS) *Баланс + кроссплатформенность*

Исходный код → Байткод → Вирт. машина → Процессор

Компромиссы

Не существует языка, лучшего во всём одновременно.

Язык	Скорость	Простота	Безопасность	Экосистема
C	●●●●●	●●●○○	●○○○○	●●○○○
C++	●●●●●	●●○○○	●●○○○	●●●○○
Java	●●●○○	●●●○○	●●●●○	●●●○○
Python	●○○○○	●●●●●	●●●○○	●●●●●
Rust	●●●●●	●○○○○	●●●●●	●●●●●
Go	●●●●○	●●●●○	●●●●○	●●●●○

Язык под задачу

Область	Популярные языки
Веб-сайты (фронтенд)	JavaScript, TypeScript
Веб-сервера (бэкенд)	Rust, Python, Java, Go, PHP, Node.js
Мобильные приложения	Kotlin (Android), Swift (iOS), Dart (Flutter)
Игры	C++, C# (Unity), GDScript (Godot)
Data Science и ML	Python, R, Julia, Matlab
Системное ПО и ОС	C, C++, Rust
Олимпиадное программирование	C++, Python
Скрипты и автоматизация	Python, Bash, PowerShell

С чего начать?

Хочу делать сайты → JavaScript
Хочу анализировать данные → Python
Хочу делать игры → C# / C++
Хочу понять компьютер → C

Важнее понять *принципы*, чем выучить синтаксис. Алгоритмы и структуры данных работают одинаково на любом языке.

Язык — это не навсегда

Второй язык изучать значительно легче, чем первый.

Концепции переносятся между языками.